

DOCKERFILE E DOCKER COMPOSE

Guia de referência para construção de imagens e definição de serviços

Material de apoio • Desenvolvimento de aplicações com containers

1. Visão geral

Dockerfile e Docker Compose são recursos relacionados, mas têm responsabilidades diferentes. O Dockerfile descreve como construir uma imagem: qual imagem base utilizar, quais arquivos copiar, quais dependências instalar e qual processo iniciar. O Docker Compose descreve como executar uma aplicação formada por um ou mais serviços, incluindo imagens, portas, variáveis, redes, volumes e dependências.

Em termos simples: o Dockerfile orienta a construção da imagem; o arquivo Compose orienta a execução e a comunicação dos serviços.

Arquivos e formatos

- Dockerfile: normalmente não possui extensão. As instruções são escritas uma por linha.
- compose.yaml ou docker-compose.yml: arquivo YAML que descreve os serviços. Os dois nomes são comuns; compose.yaml é o nome preferido na documentação atual.
- YAML usa indentação para representar hierarquia. Utilize espaços, não tabulações, e mantenha alinhamento consistente.

2. Dockerfile: instruções principais

Cada instrução do Dockerfile contribui para a construção da imagem. A ordem importa: instruções são processadas sequencialmente e podem aproveitar o cache de camadas quando seus conteúdos não mudam.

FROM — imagem base

Define a imagem sobre a qual a nova imagem será construída. Em geral, é a primeira instrução. A escolha da versão/tag influencia tamanho, compatibilidade e atualizações.

```
FROM node:22-alpine
# ou
FROM debian:trixie
```

Variações: uma tag específica facilita reproduzir o ambiente; tags como latest podem mudar com o tempo. Imagens Alpine tendem a ser menores, mas podem exigir atenção à compatibilidade de bibliotecas.

WORKDIR — diretório de trabalho

Define o diretório padrão para instruções seguintes, como RUN, COPY, ADD e CMD. Se o diretório não existir, ele será criado.

```
WORKDIR /app
```

Caminhos relativos nas instruções seguintes são interpretados a partir do WORKDIR atual.

COPY — copiar arquivos

Copia arquivos ou diretórios do contexto de construção para dentro da imagem. É a opção usual para incluir código e arquivos do projeto.

```
COPY package*.json ./
COPY . .
COPY ./config/ /app/config/
```

O contexto de construção costuma ser o diretório informado ao comando docker build ou à propriedade build do Compose. Um arquivo .dockerignore pode excluir arquivos desse contexto.

ADD — copiar com recursos adicionais

Também copia arquivos, mas oferece recursos extras, como descompactar automaticamente determinados arquivos tar locais e buscar URLs em alguns cenários. Para cópias comuns do projeto, prefira COPY por ser mais explícito.

```
ADD arquivos.tar /opt/dados/
```

Evite usar ADD apenas como sinônimo de COPY. Considere também que baixar conteúdo remoto durante a construção pode reduzir a previsibilidade da imagem.

RUN — executar comandos durante a construção

Executa comandos durante o build e registra o resultado em uma camada da imagem. É usado, por exemplo, para instalar pacotes e dependências.

```
RUN apt-get update && apt-get install -y curl
```

```
RUN npm install
```

Em imagens Debian/Ubuntu, combinar atualização da lista de pacotes e instalação na mesma instrução reduz problemas com cache. Limpar listas temporárias pode diminuir o tamanho final.

CMD — comando padrão do container

Define o comando padrão executado quando um container é iniciado. Pode ser substituído por argumentos informados no comando docker run ou pela configuração do Compose.

```
CMD ["npm", "start"]
```

A forma em lista (exec form) é geralmente preferível: evita uma camada de shell intermediária e trata melhor sinais enviados ao processo. Normalmente há uma instrução CMD efetiva; se houver várias, a última prevalece.

ENTRYPOINT — processo principal

Define o executável principal do container. É útil quando a imagem deve sempre executar um programa específico, enquanto CMD pode fornecer argumentos padrão.

```
ENTRYPOINT ["python", "app.py"]
```

```
CMD ["--port", "8080"]
```

ENTRYPOINT e CMD podem ser combinados. A forma de lista é recomendada na maioria dos casos. No Compose, entrypoint e command também podem sobrescrever essas definições.

EXPOSE — documentar a porta

Documenta a porta em que a aplicação costuma escutar dentro do container. Não publica a porta automaticamente para a máquina host.

```
EXPOSE 3000
EXPOSE 80/tcp
```

Para permitir acesso externo, a publicação normalmente é configurada com ports no Compose ou -p no docker run.

ENV — variáveis de ambiente

Define variáveis disponíveis durante a construção e também nos containers criados a partir da imagem, salvo substituição em tempo de execução.

```
ENV NODE_ENV=production
ENV APP_PORT=3000
```

MATERIAL DE APOIO • CONTAINERS E ORQUESTRAÇÃO

Não grave senhas, tokens ou chaves privadas em ENV no Dockerfile: esses valores podem ficar registrados na imagem. Utilize mecanismos apropriados de configuração e secrets.

ARG — variável de build

Define uma variável disponível durante a construção da imagem. Pode receber valor por `--build-arg`. Não é, por si só, um mecanismo seguro para segredos.

```
ARG NODE_VERSION=22
FROM node:${NODE_VERSION}-alpine
```

ARG tem escopo relacionado ao build; ENV é destinada a variáveis presentes no ambiente do container.

USER — usuário do processo

Define o usuário e, opcionalmente, o grupo usados nas instruções seguintes e no processo padrão do container. Rodar aplicações como usuário não-root reduz privilégios.

```
USER node
```

Antes de trocar de usuário, verifique permissões de leitura, escrita e execução dos arquivos necessários.

VOLUME — ponto de montagem

Declara um caminho destinado a armazenamento persistente ou compartilhado. A declaração não define, sozinha, o nome de um volume gerenciado; o vínculo concreto pode ser configurado na execução ou no Compose.

```
VOLUME ["/var/lib/mysql"]
```

Para dados que precisam sobreviver à substituição de containers, configure volumes explicitamente e entenda seu ciclo de vida.

LABEL — metadados

Adiciona metadados à imagem, como descrição, versão, responsável ou endereço do projeto.

```
LABEL description="API de demonstração"
LABEL version="1.0"
```

SHELL e HEALTHCHECK — instruções complementares

SHELL altera o shell usado por instruções em formato shell em determinadas plataformas. HEALTHCHECK define um teste periódico para indicar se o processo dentro do container está saudável.

```
SHELL ["/bin/bash", "-c"]

HEALTHCHECK --interval=30s --timeout=5s --retries=3 \\\
  CMD curl -f http://localhost:3000/ || exit 1
```

HEALTHCHECK é opcional. O teste deve verificar uma condição útil e disponível na imagem. Um container em execução não é necessariamente um serviço pronto.

3. Docker Compose: estrutura YAML

O Compose reúne a configuração dos serviços de uma aplicação. A estrutura mais comum contém services e, quando necessário, volumes, networks e secrets no nível principal.

```
services:
  aplicacao:
```

```
    image: exemplo:1.0

volumes:
  dados:

networks:
  rede-interna:
```

A indentação define quais propriedades pertencem a cada serviço. Por exemplo, ports dentro de um serviço configura aquele serviço; volumes no nível principal declara recursos nomeados que podem ser referenciados pelos serviços.

services — serviços da aplicação

Cada chave dentro de services representa um serviço lógico. O nome do serviço também pode ser usado como nome DNS pelos outros serviços conectados à mesma rede Compose.

```
services:
  backend:
    image: minha-api:1.0
  db:
    image: mysql:8.4
```

Um serviço pode usar uma imagem existente ou construir uma imagem a partir de um Dockerfile.

image e build — origem do container

image indica a imagem a utilizar. build informa que o Compose deve construir uma imagem a partir de um contexto, podendo apontar para um Dockerfile.

```
services:
  web:
    image: nginx:alpine

  api:
    build: ./backend

  worker:
    build:
      context: ./worker
      dockerfile: Dockerfile.dev
```

Também é possível definir args de build em build.args. O contexto precisa conter os arquivos que o Dockerfile referencia.

container_name — nome explícito

Define um nome fixo para o container. É útil em demonstrações, mas pode dificultar executar várias cópias do mesmo projeto, pois nomes de container precisam ser únicos no host.

```
container_name: api-demonstracao
```

ports — publicação de portas

Mapeia uma porta do host para uma porta do container. A forma curta é HOST:CONTAINER. A porta interna é aquela em que o processo escuta dentro do container.

```
ports:
  - "8080:80"
  - "127.0.0.1:3306:3306"
```

O segundo exemplo limita a publicação ao loopback do host. Sem um endereço explícito, a porta pode ser publicada em interfaces de rede do host. Evite expor bancos de dados sem necessidade e sem controles de acesso.

expose — portas internas

Documenta portas que devem estar acessíveis a outros serviços conectados, sem publicá-las diretamente no host. Em redes Compose, serviços normalmente conseguem se comunicar pela porta interna sem declarar expose.

```
expose:
  - "3000"
```

environment e env_file — configuração

environment define variáveis diretamente no arquivo Compose. env_file carrega variáveis de um arquivo externo. São formas de configurar o ambiente do serviço.

```
environment:
  APP_MODE: desenvolvimento
  APP_PORT: 3000

# alternativa
env_file:
  - .env
```

Ajuste a indentação conforme o nível do serviço. Arquivos .env não devem ser enviados ao repositório quando contêm credenciais. Para ambientes sensíveis, avalie secrets ou um gerenciador de segredos.

volumes — persistência e compartilhamento

Volumes permitem manter dados fora do ciclo de vida do container ou compartilhar arquivos entre host e container. Há volumes nomeados e bind mounts.

```
services:
  db:
    volumes:
      - dados_mysql:/var/lib/mysql
      - ./scripts:/docker-entrypoint-initdb.d:ro

volumes:
  dados_mysql:
```

No formato curto, origem:destino. Uma origem nomeada é declarada no nível principal; um caminho como ./scripts é um diretório do host. O sufixo :ro monta como somente leitura.

Remover containers não implica necessariamente remover volumes. O comando docker compose down -v remove também os volumes nomeados declarados pelo projeto e pode apagar dados.

depends_on — ordem e condição

Define dependências entre serviços. A forma simples organiza a ordem de inicialização, mas não garante que o serviço dependente esteja pronto para receber requisições. Com condition: service_healthy, o Compose aguarda o healthcheck do serviço dependido.

```
depends_on:
  db:
    condition: service_healthy
```

A condição `service_healthy` depende de um `healthcheck` definido no serviço referenciado. Mesmo assim, aplicações devem tratar falhas temporárias de rede e reconexões.

healthcheck — verificação de saúde

Executa periodicamente um teste dentro do container e classifica o serviço como `starting`, `healthy` ou `unhealthy`. O teste deve retornar código zero quando a condição verificada for bem-sucedida.

```
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
  interval: 5s
  timeout: 5s
  retries: 10
  start_period: 20s
```

`test` pode usar `CMD` ou `CMD-SHELL`. `interval` define o intervalo entre verificações; `timeout` limita a duração de cada teste; `retries` define falhas consecutivas para marcar `unhealthy`; `start_period` concede tempo inicial de tolerância. Um teste superficial pode indicar saúde sem comprovar que todas as operações da aplicação funcionarão.

restart — política de reinício

Define em quais situações o Docker deve reiniciar um container.

```
restart: no
restart: always
restart: on-failure
restart: unless-stopped
```

`no` não reinicia automaticamente; `always` reinicia quando o container termina; `on-failure` considera término com erro; `unless-stopped` reinicia, exceto quando foi parado explicitamente. Políticas de reinício não substituem monitoramento nem correção de falhas.

networks — redes

Permite definir redes e conectar serviços a elas. O Compose cria uma rede padrão para o projeto quando nenhuma configuração específica é fornecida. Serviços na mesma rede podem localizar uns aos outros pelo nome do serviço.

```
services:
  api:
    networks:
      - interna
  db:
    networks:
      - interna

networks:
  interna:
```

Dentro de um container, `localhost` aponta para o próprio container, não para outro serviço. Para comunicação entre serviços, utilize o nome DNS do serviço e a porta interna.

command e entrypoint — sobrescrever execução

`command` substitui o `CMD` da imagem; `entrypoint` substitui o `ENTRYPOINT`. Podem ser usados para ajustar o processo de um serviço sem alterar o `Dockerfile`.

```
command: ["npm", "start"]
# ou
command: sh -c "python manage.py migrate && python app.py"
```

A forma em lista é mais previsível. Use shell apenas quando precisar de recursos do shell, como encadeamento de comandos.

4. Variáveis e substituição no Compose

O Compose permite interpolar variáveis no arquivo YAML usando `${NOME}`. Os valores podem vir do ambiente do terminal ou de um arquivo `.env`, conforme as regras de carregamento do Compose.

```
services:
  api:
    ports:
      - "${PORTA_HOST:-3000}:3000"
```

A expressão `:-` define um valor padrão quando a variável não está definida ou está vazia. Isso ajuda a reutilizar o mesmo arquivo em diferentes ambientes. Não confunda interpolação do Compose com `environment`: a primeira substitui valores no YAML; a segunda define variáveis dentro do container.

5. Comandos úteis para trabalhar com Compose

Os comandos abaixo são executados no diretório que contém o arquivo Compose, salvo quando se informa outro caminho com `-f`.

<code>docker compose config</code>	Valida e apresenta a configuração final após interpolação de variáveis.
<code>docker compose up -d</code>	Cria e inicia os serviços em segundo plano.
<code>docker compose up -d --build</code>	Constrói ou reconstrói imagens antes de iniciar os serviços.
<code>docker compose ps</code>	Lista os serviços e seus estados.
<code>docker compose logs -f</code>	Acompanha os logs dos serviços em tempo real.
<code>docker compose logs -f nome_servico</code>	Acompanha os logs de um serviço específico.
<code>docker compose exec nome_servico sh</code>	Abre um shell em um container em execução, se a imagem tiver sh.
<code>docker compose stop</code>	Para os containers sem removê-los.
<code>docker compose start</code>	Inicia containers que já existem e estão parados.
<code>docker compose restart</code>	Reinicia os serviços.
<code>docker compose down</code>	Para e remove containers e a rede criada pelo projeto.
<code>docker compose down -v</code>	Também remove volumes nomeados do projeto; pode apagar dados persistentes.
<code>docker compose pull</code>	Baixa imagens declaradas, sem necessariamente iniciar serviços.
<code>docker compose build</code>	Constrói as imagens dos serviços que usam build.

6. Boas práticas e cuidados

- Prefira tags de imagem explícitas em vez de depender de `latest` quando a reprodução do ambiente for importante.
- Mantenha segredos fora do Dockerfile e do repositório. Use arquivos de ambiente protegidos, `secrets` ou soluções próprias de gerenciamento.
- Não confunda `EXPOSE` com `ports`: `EXPOSE` documenta; `ports` publica uma porta no host.
- Não confunda `depends_on` simples com prontidão: use `healthcheck` quando fizer sentido e implemente tolerância a falhas na aplicação.
- Use `.dockerignore` para evitar enviar ao contexto arquivos desnecessários, como `node_modules`, `.git`, arquivos temporários e segredos.

- Use volumes para dados que precisam persistir. Antes de executar `down -v`, confirme que os dados podem ser descartados.
- Evite executar processos como root quando a imagem e a aplicação permitirem um usuário com privilégios limitados.
- Em produção, não exponha portas de banco de dados publicamente sem necessidade. Restrinja interfaces, firewall e permissões.
- Prefira COPY a ADD para cópias simples. Prefira a forma `exec` em CMD e ENTRYPOINT.
- Consulte os logs e o estado dos serviços antes de alterar configurações; mensagens de erro ajudam a localizar a camada com problema.

7. Referência rápida: conceitos que não devem ser confundidos

Imagem	Modelo imutável usado para criar containers.
Container	Instância em execução (ou parada) criada a partir de uma imagem.
Build	Processo de construção de uma imagem, normalmente usando um Dockerfile.
Serviço Compose	Definição lógica de como um componente da aplicação deve ser executado.
Volume	Recurso de armazenamento gerenciado ou montado, usado para persistência/compartilhamento.
Rede	Meio de comunicação entre containers.
Healthcheck	Teste que informa uma condição de saúde; não substitui testes funcionais completos.

8. Documentação oficial

Para consultar sintaxe atualizada, exemplos e opções específicas de cada versão:

- Dockerfile reference: <https://docs.docker.com/reference/dockerfile/>
- Docker Compose file reference: <https://docs.docker.com/reference/compose-file/>
- Docker Compose overview: <https://docs.docker.com/compose/>
- Docker build context e .dockerignore: <https://docs.docker.com/build/concepts/context/>

Fim do material